



OASIS Security Report

Command Injection Security Analysis

Date: 2026-06-11 22:59:17 — **Model:** llama3.2:3b

Vulnerability: Command Injection

Project: vulnerable

Analysis root: `../vulnerable`

Table of contents

▢ Summary

▢ Statistics

▢ Files

▢ Detailed analysis

▢ Assistant

△ Errors & notes

Summary

Analyzed **6** file(s) for Command Injection.

[Return to table of contents](#)

Statistics

Files analyzed: 6 **Total findings:** 9



Medium

3

Low

0

[Return to table of contents](#)

Files

File	Similarity
/work/project/dataset/vulnerable/reentrancy/simple_dao.sol	0.599
/work/project/dataset/vulnerable/reentrancy/ 0xbaf51e761510c1a11bf48dd87c0307ac8a8c8a4f.sol	0.599
/work/project/dataset/vulnerable/access-control/parity_wallet_bug_1.sol	0.594
/work/project/dataset/vulnerable/reentrancy/etherstore.sol	0.588
/work/project/dataset/vulnerable/access-control/parity_wallet_bug_2.sol	0.575
/work/project/dataset/vulnerable/ordering-attacks/odds_and_evens.sol	0.571

[Return to table of contents](#)

Detailed analysis

File 1: /work/project/dataset/vulnerable/reentrancy/simple_dao.sol

Similarity score: 0.599

Source lines: 1-35

► Finding 1.1: Command Injection (Critical)

Vulnerable code

```
bool res = msg.sender.call.value(amount)();
```

Explanation

Using `msg.sender.call()` with user input without proper validation allows command injection.

Impact

Complete system compromise, data theft, privilege escalation, or service disruption

Entry point

`withdraw(uint amount)`

Execution path

```
call -> sendValue -> exec
|         |
|  value  |
|         |
|         | \
|         | /\
|         | / \
|         v   \
```

Parameters

amount

Exploitation steps

- Send a large amount to trigger command injection

Example payloads

- `\x00\x0aecho Welcome\x10!`

Conditions

msg.sender is authenticated

Remediation

Use proper validation and sanitization for user input, e.g., using a whitelist of allowed characters.

Secure example

```
bool res = msg.sender.call.value(amount, "echo \x0aHello\x10!\n")();
```

File 2: /work/project/dataset/vulnerable/reentrancy/0xbaf51e761510c1a11bf48dd87c0307ac8a8c8a4f.sol

Similarity score: 0.599

Source lines: 1-85

► Finding 2.1: Vulnerability found (Critical)

Vulnerable code

```
function CashOut(uint _am) public payable { if(_am<=balances[msg.sender])  
{ msg.sender.call.value(_am)(); } }
```

Explanation

The call function without proper input validation allows an attacker to execute system commands by manipulating the _am value.

Impact

Complete system compromise, data theft, privilege escalation, or service disruption

Entry point

/CashOut

Execution path

```
Call function with user-provided _am value -> Execution of system command ->  
Potential system compromise
```

Parameters

_am

Exploitation steps

- User sends malicious _am value to CashOut

Example payloads

- ; ls -l
- ; id

Conditions

Authenticated user with access to msg.sender's balances

Remediation

Validate and sanitize the _am value before executing the call function.

Secure example

```
function CashOut(uint _am) public payable { require(_am > 0, "Invalid amount"); balances[msg.sender] -= _am; }
```

File 3: /work/project/dataset/vulnerable/access-control/parity_wallet_bug_1.sol

Similarity score: 0.594

Source lines: 126-467

► Finding 3.1: Command Injection via delegatecall (Critical)

Vulnerable code

```
function hasConfirmed(bytes32 _operation, address _owner) external constant  
returns (bool) { return _walletLibrary.delegatecall(msg.data); }
```

Explanation

The use of delegatecall allows an attacker to inject arbitrary operating system commands by manipulating the input to msg.data.

Impact

Complete system compromise, data theft, privilege escalation, or service disruption

Entry point

/hasConfirmed

Execution path

```
delegatecall -> hasConfirmed() -> delegatecall(msg.data) -> unknown code  
execution
```

Parameters

msg.data

Exploitation steps

- Manipulate msg.data to inject commands

Example payloads

- `echo 'Hello, World!' > file.txt`

Conditions

msg.sender has permission to call delegatecall

Remediation

Use safe fallback functions instead of delegatecall

Secure example

```
function hasConfirmed(bytes32 _operation, address _owner) external constant
returns (bool) { return true; }
```

Source lines: 207-477

► Finding 3.1: Command Injection Vulnerability (Critical)

Vulnerable code

```
if (!_to.call.value(_value)(_data)) throw;
```

Explanation

The `\_to.call.value(\_value)(\_data)` call is vulnerable to Command Injection because it executes user-provided input as a system command.

Impact

Complete system compromise, data theft, privilege escalation, or service disruption

Entry point

/execute(address _to, uint _value, bytes _data) external onlyowner returns (bytes32 o_hash)

Execution path

```

    ^
    |
execute
  call
    value
      _value
      (_data)
    ^
    |
    ^
```

Parameters

_to, _value, _data

Exploitation steps

- Send a crafted value and data to the execute function to bypass validation

Example payloads

- `echo Hello World!`

Conditions

Unauthenticated users can reach the execute function

Remediation

Validate and sanitize user-provided input before passing it to ``_to.call.value(_value)(_data)``, using a whitelist of allowed commands and parameters.

Secure example

```
if (!_to.call.value(_value, 'echo', _data)) throw;
```

File 4: /work/project/dataset/vulnerable/reentrancy/etherstore.sol

Similarity score: 0.588

Source lines: 1-39

► Finding 4.1: Command Injection (Critical)

Vulnerable code

```
require(msg.sender.call.value(_weiToWithdraw)());
```

Explanation

The `msg.sender.call()` function allows injection of arbitrary operating system commands.

Impact

Complete system compromise, data theft, privilege escalation, or service disruption

Entry point

`withdrawFunds` function

Execution path

```
msg.sender.call() -> executes system command
```

Parameters

`_weiToWithdraw`, `msg.sender`

Exploitation steps

- Send a crafted value for `_weiToWithdraw` to inject shell commands.
- Trigger `msg.sender.call()` with the malicious input.

Example payloads

- `echo 'Hello World!' | /bin/ls -l`

HTTP raw requests (Burp / ZAP)

```
POST /withdrawFunds HTTP/1.1
```

```
Host: example.com
```

Content-Type: application/json

```
{"weiToWithdraw": "echo 'Hello World!' | /bin/ls -l"}
```

File 5: /work/project/dataset/vulnerable/access-control/parity_wallet_bug_2.sol

Similarity score: 0.575

Source lines: 1-286

► Finding 5.1: Command Injection (High)

Vulnerable code

```
function execute(address _to, uint _value, bytes _data) external onlyowner
returns (bytes32 o_hash) {
    // first, take the opportunity to check that we're under the daily limit.
    if ((_data.length == 0 && underLimit(_value)) || m_required == 1) {
        // yes - just execute the call.
        address created;
        if (_to == 0) {
            created = create(_value, _data);
        } else {
            if (!_to.call.value(_value)(_data)) throw;
        }
        SingleTransact(msg.sender, _value, _to, _data, created);
    } else {
        // determine our operation hash.
        o_hash = sha3(msg.data, block.number);
        // store if it's new
        if (m_txs[o_hash].to == 0 && m_txs[o_hash].value == 0 &&
m_txs[o_hash].data.length == 0) {
            m_txs[o_hash].to = _to;
            m_txs[o_hash].value = _value;
            m_txs[o_hash].data = _data;
        }
        if (!confirm(o_hash)) {
            ConfirmationNeeded(o_hash, msg.sender, _value, _to, _data);
        }
    }
}
```

Explanation

The `execute` function uses `msg.data` directly, making it vulnerable to Command Injection attacks.

Impact

An attacker could inject malicious commands and potentially take control of the contract.

Entry point

execute(address _to, uint _value, bytes _data)

Parameters

_data

Exploitation steps

- Pass malicious data to the `execute` function

Example payloads

- `echo 'Hello World!' | base64`

Conditions

The contract is executed with elevated privileges.

Remediation

Use parameterized commands and validate user input

Secure example

```
function execute(address _to, uint _value, bytes _data) external onlyowner
returns (bytes32 o_hash) {
    bytes data = ethers.utils.parseBytes-like(msg.data);
    // ...}
```

Source lines: 96-414

► Finding 5.1: Potential Command Injection (Medium)

Vulnerable code

```
function changeOwner(address _from, address _to)
onlymanyowners(sha3(msg.data)) external {
    if (isOwner(_to)) return;
    uint ownerIndex = m_ownerIndex[uint(_from)];
    if (ownerIndex == 0) return;

    clearPending();
    m_owners[ownerIndex] = uint(_to);
    m_ownerIndex[uint(_from)] = 0;
    m_ownerIndex[uint(_to)] = ownerIndex;
    OwnerChanged(_from, _to);
}
```

Explanation

The `sha3(msg.data)` call allows an attacker to inject arbitrary shell commands through the `\_from` parameter.

Impact

Privilege escalation or command execution

Entry point

changeOwner(address _from, address _to)

Parameters

_from, _to

Exploitation steps

- Use a crafted `\_from` value to inject shell commands (e.g., `; ls -l`).

Example payloads

- ```
; echo 'Hello, World!' > file.txt
```

**Conditions**

The `\_from` parameter is not sanitized.

**Remediation**

Use parameterized commands and validate the input data.

**Secure example**

```
function changeOwner(address _from, address _to) onlymanyowners(_operation)
external {
 // ...
}
```

## File 6: /work/project/dataset/vulnerable/ordering-attacks/odds\_and\_evens.sol

Similarity score: 0.571

Source lines: 1-61

### ► Finding 6.1: Command Injection (Medium)

#### Vulnerable code

```
function play(uint number) payable{
 if (msg.value != 1 ether) throw;
 players[tot] = Player(msg.sender, number);
 tot++;
}
```

#### Explanation

### ► Finding 6.2: Command Injection (Medium)

#### Explanation

[Return to table of contents](#)

## Assistant (triage / codebase)

In the OASIS dashboard (JSON report preview), use the chat below to triage findings, exploitation paths, and remediation. Select file, chunk, and finding to narrow context when needed.

[Return to table of contents](#)

## Errors & Notes

No file-level errors were recorded.

[Return to table of contents](#)